FINM 32950: Intro to HPC in Finance

Lecture 4

Chanaka Liyanarachchi

April 15, 2026

Black Scholes Pricer

Monte Carlo on the GPU

Using GPUs in Python

Course Wrap-up

Black Scholes Pricer

Black Scholes Pricer

- You can structure this program very similar to the previous vector additon example.
- How do we write the kernel function (to price Options)?
  - Inputs: arrays of S, K, T, r, v
  - Output: call price (ignoring put price and greeks)

```
__global__ void BlackScholesKernel(
    float *S, float *K, float *T,
    float* r, float* v,
    float* C, int N)
```

- For each option, we calculate:

```
    d1 = ...;
    d2 = ...;
    nd1 = cdf_normal(d1);
    nd2 = cdf_normal(d2);
```

- Now we can find the call price and the put price:

```
    C[idx] = ...;
    P[idx] = ...;
}
```

- A kernel function can use other device functions.
- E.g. `cdf_normal()`

Device Functions

- `cdf_normal()` is called from the GPU and executed on the GPU.
- We define this function using `__device__` qualifier:

```
__device__ float cdf_normal(const float x)
{
    .....
    .....
}
```

- This kernel and device functions use several math functions.
- Reference: CUDA Math functions
  `https://docs.nvidia.com/cuda/cuda-math-api/index.html`

Assignment G

- Populate all input data on the CPU (as in Assignment 2).
- Transfer inputs to the GPU, perform all computations on the GPU, and copy the results back to the CPU.
- Additional (optional) exercise: implement a tree-based option pricer using CUDA.

Monte Carlo on the GPU

Generating Random Numbers

- We can use the cuRAND[1] library to generate random numbers in CUDA applications.
- Code below shows how to generate pseudo random numbers from the uniform distribution:
- Step 1: create a generator. It takes a generator type as a argument.

```
curandGenerator_t gen;

curandCreateGenerator(&gen, CURAND_RNG_PSEUDO_MTGP32);
```

- Step 2: initialize it using a seed:

```
curandSetPseudoRandomGeneratorSeed(gen, 1234ULL);
```

[1] http://docs.nvidia.com/cuda/curand/index.html#axzz4jskVkPyT

- Step 3: allocate memory on the device:

```
int N = 1000;
int numBytes = N*sizeof(float);

float *d_rands;
cudaMalloc(&d_rands, numBytes);
```

- Step 4: generate N random numbers on the device:

```
curandGenerateUniform(gen, d_rands, N);
```

- cuRAND supports random number generation from the following distributions:
  1. uniform
  2. normal
  3. lognormal
  4. poisson
- Reference: `http://docs.nvidia.com/cuda/curand/device-api-overview.html#distributions`

- Now we can use these random numbers on the device.
- Cleaning up:

```
curandDestroyGenerator(gen);
```

- And, clean up device memory:

```
cudaFree(d_rands);
```

- See curand_test.cu for details.
- To build we need to pass curand library as an additional argument:

```
nvcc -lcurand curand_test.cu -o curand_test
```

- To run:

```
./curand_test
```

CUDA Example: Monte Carlo PI

- We've estimate the value of π using Monte Carlo.
- Used serial code and parallel code on CPU.
- The Code below shows how to do this simulation on the GPU.

- MC PI kernel (Basic):
  - Uses N points (each point is a pair of random numbers from the uniform distribution). You could use one array (x and y) or two arrays, as shown below
  - result: indicator if each point is in the circle

```
__global__ void pi_kernel(float* x,
           float* y,
           int* results, int N)
{
```

- Thread index:

```
   int idx = threadIdx.x + blockDim.x * blockIdx.x;
```

- Pick x and y points and compute the radius:

```
   float radius = x[idx]*x[idx] + y[idx]*y[idx];
```

- Using the radius we can determine if the point is inside the circle.

```
   if (radius <= 1) results[idx] = 1;
   else  results[idx] = 0;
}
```

- See demo for details.

Assignment 3 (Required)

- Details Later

Using GPUs in Python

We have several solutions to use GPUs in Python.

- CuPy
- cuPyNumeric
- Numba
- Cython
- ...

CuPy

What is CuPy?

- Python library for GPU computing
- NumPy-like interface
- Runs on NVIDIA GPUs using CUDA

Key idea:

- Same code style as NumPy
- Computation executed on GPU
- Drop-in replacement for many NumPy workflows
- Minimal changes required; familiar syntax preserved
- `https://github.com/cupy/cupy`

CuPy: Environment Setup

```
module load cuda/12.0
module load python

python3 -m venv .venv
source .venv/bin/activate

pip install cupy-cuda12x
```

CuPy Example: Creating Arrays

- `cupy.ndarray` stored on GPU

```
import cupy as cp

x = cp.array([1, 2, 3])
y = cp.zeros((2, 3))
z = cp.ones_like(y)
```

- Supports indexing, broadcasting, reshaping, and dtypes

CuPy Example: Matrix Multiplication

- Some matrix operations:

```
matrix1 = cp.array([[1, 2], [3, 4]])
matrix2 = cp.array([[5, 6], [7, 8]])

mat_dot = cp.dot(matrix1, matrix2)
mat_elem = cp.multiply(matrix1, matrix2)
mat_at = matrix1 @ matrix2

print(mat_dot)
print(mat_elem)
print(mat_at)
```

- `cp.dot()` and `@` perform matrix multiplication
- `cp.multiply()` does element-wise multiplication

CuPy: CPU and GPU Memory Copy

- Move data CPU $\rightarrow$ GPU: `cp.asarray()`
- Move data GPU $\rightarrow$ CPU: `cp.asnumpy()` or `.get()`
- Minimize CPU-GPU transfers for performance

Random Number Generation

- GPU-based RNG using cuRAND

```
rng = cp.random.default_rng()
x = rng.normal(size=1000000)
```

- All computations remain on GPU

CuPy Example: Elementwise Kernel

```
square = cp.ElementwiseKernel(
    'float32 x', 'float32 y',
    'y = x * x;', 'square'
)
data = cp.arange(5, dtype=cp.float32)
out = square(data)
```

- The string `'y = x * x;'` is compiled into actual CUDA C++ code and cached for future use.
- GPU threads are launched; each thread handles exactly one element simultaneously.
- Applies the operation to each element independently
- The Workflow
  1. Input: `x` reads from `data` in GPU memory
  2. Output: Result is written directly to the `y` array.
- Highly parallel on GPU

CuPy Example: Reduction Kernel

```
sum_kernel = cp.ReductionKernel(
    'float32 x', 'float32 y',
    'x', 'a + b', 'y = a', '0', 'sum_kernel'
)
x = cp.arange(5, dtype=cp.float32)
result = sum_kernel(x)
```

- Combines elements into a single result (sum, max, etc.)
- Requires synchronization across threads

Kernel with Device Function

```
square_kernel = cp.ElementwiseKernel(
    'float32 x', 'float32 y',
    'y = my_square(x);', 'square_kernel',
    preamble='''
    __device__ float my_square(float v) {
        return v * v;
    }
    '''
)
x = cp.array([2.0], dtype=cp.float32)
out = square_kernel(x)
print(out)
```

Kernel with Preamble (Device Functions)

```
square_kernel = cp.ElementwiseKernel(
    'float32 x', 'float32 y',
    'y = my_square(x);', 'square_kernel',
    preamble='''
    __device__ float my_square(float v) {
        return v * v;
    }
    '''
)
x = cp.array([1, 2, 3], dtype=cp.float32)
out = square_kernel(x)
```

- The Preamble: A string of raw CUDA C++ code prepended to the generated kernel before compilation.
- `__device__` qualifier: Tells the compiler this function is callable only by other functions on the GPU.

CuPy Finance Examples

- Finance Examples:
  https://github.com/cupy/cupy/blob/main/examples/

```
[chanaka@midway3-0279 cupy]$ module list
Currently Loaded Modulefiles:
 1) slurm/current   2) rcc/default   3) cuda/12.0
[chanaka@midway3-0279 cupy]$ module load python
[chanaka@midway3-0279 cupy]$ python3 -m venv .venv
[chanaka@midway3-0279 cupy]$ source .venv/bin/activate
(.venv) [chanaka@midway3-0279 cupy]$ pip install cupy-cuda12x
Collecting cupy-cuda12x
  Using cached cupy_cuda12x-13.6.0-cp39-cp39-manylinux2014_x86_64.whl (112.3 MB)
Collecting fastrlock≥0.5
  Using cached fastrlock-0.8.3-cp39-cp39-manylinux_2_5_x86_64.manylinux1_x86_64.manylinux_2_28_x86_64.whl (55 kB)
Collecting numpy<2.6,≥1.22
  Using cached numpy-2.0.2-cp39-cp39-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (19.5 MB)
Installing collected packages: fastrlock, numpy, cupy-cuda12x
Successfully installed cupy-cuda12x-13.6.0 fastrlock-0.8.3 numpy-2.0.2

[notice] A new release of pip is available: 23.0.1 → 26.0.1
[notice] To update, run: pip install --upgrade pip
(.venv) [chanaka@midway3-0279 cupy]$ python black_scholes.py
initializing...
start computation
 CPU (NumPy, Naive implementation):     1.107749 sec
 GPU (CuPy, Naive implementation):      2.896723 sec
 GPU (CuPy, Elementwise kernel):        0.221364 sec
(.venv) [chanaka@midway3-0279 cupy]$
```

CuPy: Key Takeaways

- CuPy enables GPU computing in Python
- Each operation launches a kernel
- Custom kernels improve performance
- Same principles as CUDA C++

Other Options

Other Python Options for CUDA

- Cython:
  - Allows calling parallel algorithms from the C++ Standard Library on GPUs.
  - Example: `https://developer.nvidia.com/blog/accelerating-standard-c-with-gpus-using-stdpar/`
  - Requires knowledge of C++ features.
- Numba:
  - Enables writing pure Python code for CPU and GPU execution.
  - Compiles code at runtime, supporting NVIDIA and AMD GPUs on Windows, Linux, and macOS.
  - Reference: `https://numba.pydata.org/`
- There are other tools; explore as needed.

What is Numba?

- `https://numba.readthedocs.io/en/stable/`
- Python library for Just-In-Time (JIT) compilation
- Translates Python functions into fast machine code
- Can target CPU and GPU
- Numba CUDA `https://nvidia.github.io/numba-cuda/`

Key idea:

- Write Python
- Execute at near C/CUDA speed

Why Use Numba?

- Python is easy to write but can be slow
- GPUs require parallel execution
- Numba bridges Python and high-performance computing

Use cases:

- Numerical simulations
- Monte Carlo methods
- Custom GPU kernels

Numba for GPU

- Numba can generate CUDA kernels
- Uses Python syntax to express parallelism
- Similar model to CUDA C++
- `https://nvidia.github.io/numba-cuda/user/installation.html#numba-cuda-installation`

Key idea:

- One thread handles one element

CUDA Vector Addition (Numba)

```
@cuda.jit
def add_vector(a, b, c):
    tid = cuda.grid(1)
    size = len(c)

    if tid < size:
        c[tid] = a[tid] + b[tid]
```

- Each thread computes one element of the output vector
- Threads operate independently (data-parallel pattern)

Thread Indexing

- Global thread index:

```
tid = threadIdx.x + blockIdx.x * blockDim.x
```

- In Numba:

```
tid = cuda.grid(1)
```

- Ensures each thread works on a unique element
- Bounds check is required:

```
if tid < size:
```

- Prevents out-of-bounds memory access

Kernel Launch: High-Level

```
add_vector.forall(N)(a, b, c)
```

- Convenience interface provided by Numba
- Automatically determines grid and block sizes
- Conceptually launches one thread per element
- Useful for simple data-parallel problems
- Limited control over performance tuning

Kernel Launch: Manual Configuration

```
nthreads = 256
nblocks = (N + nthreads - 1) // nthreads
add_vector[nblocks, nthreads](a, b, c)
```

- Explicit control over:
  - Threads per block
  - Number of blocks
- Standard CUDA execution model
- Allows performance tuning
- Same kernel, different launch strategy

Memory Management

```
d_a = cuda.to_device(a)
d_b = cuda.to_device(b)
d_c = cuda.device_array_like(a)

add_kernel[blocks, threads](d_a, d_b, d_c)

c = d_c.copy_to_host()
```

- Explicit CPU to GPU transfers
- More control than high-level libraries
- Observe how closely Numba follows the CUDA C programming model.

Assignment 3 (Required)

- Implement a GPU-accelerated program to price European call and put options under the Black–Scholes framework using Monte Carlo simulation.
- The implementation must run on a GPU.
- You may use CUDA or any GPU-enabled Python framework (e.g., CuPy, Numba, or any other).
- Full assignment details will be provided on Canvas.

Course Wrap-up

We Discussed

- This course is about to improve performance using parallel programming.
- We discussed 3 main technologies:
  1. Vectorization
  2. Multicore
  3. GPGPU
- We discussed how to optimize and improve performance using a structured approach:
  - measure
  - profile
  - experiment meaningful changes
  - repeat

Vectorization

- We can get significant speed-ups on modern processors using SIMD:
  - we can use intrinsics
  - or, auto vectorization
- We looked at both techniques.
- We prefer auto vectorization:
  - we have to understand when a compiler vectorizes code and when it does not
  - how to guide to the compiler to vectorize code
  - read vectorization reports

Multicore

- We discussed basic ideas about multithreading:
  - using threads to do work in parallel
  - related concepts: critical sections, race conditions, locks, deadlocks, atomics
- Solve problems in finance using high level constructs:
  - OpenMP
  - TBB
- Introduced structured parallelism, using parallel patterns.

GPGPU

- Not an easy topic for a beginner-level course in a short time.
- We looked at some useful applications.
- We understand the power of CUDA and its limitations.
- There are many other interesting and important topics we can look at, but we have to stop here due to time constraints.

Example Applications

- Parallel programming is a broad topic.
- There are many possible approaches and tools.
- Examples we have looked at include:

Example	C++ Serial	C++ MT	Python/Cython	CUDA (GPU)	Python + GPU
Black-Scholes	✓	✓	✓	✓	✓
Tree Pricer	✓	try it	try it	try it	try it
MC PI	✓	✓	try it	✓	try it
MC Pricer	✓	try it	try it	✓	✓

- This provides an opportunity to learn each technology and observe trade-offs, strengths, and limitations firsthand.

- Learning the trade-offs of each technology allows you to select the most appropriate tool for your specific use case and your comfort level.
- It is unrealistic to anticipate that everyone will master these concepts within a span of 4 weeks; acquiring proficiency requires time and practice.
- The Course objective was to provide an introduction to essential technologies, demonstrating their possibilities and initiating your journey. I trust we have met that objective.

Next ...

- Apply HPC concepts when you get an opportunity.
- Practice and keep learning.
- Please don't hesitate to contact me if you need any assistance.

THANK YOU

PLEASE KEEP IN TOUCH

GOOD LUCK!